

Principle and Practice of Bidirectional Programming in BiGUL

Zhenjiang Hu, Hsiang-Shang Ko

National Institute of Informatics, Japan
{hu,hsiang-shang}@nii.ac.jp

Abstract. This material will be used for the Summer School on Bidirectional Transformation to be held in Oxford in July, 2016.

1 Installation

The latest version of BiGUL has been released to Hackage. To install the package, just run the following two commands.

```
> cabal update
> cabal install bigul
```

Now you can easily check whether BiGUL is correctly installed. First, create a simple file called “test.hs” with the following content for importing BiGUL modules.

```
import Generics.BiGUL.Error
import Generics.BiGUL.AST
import Generics.BiGUL.Interpreter
import Generics.BiGUL.TH
```

Then load it using the GHC interactive interpreter.

```
> ghci test.hs
GHCi, version 7.10.3: http://www.haskell.org/ghc/  :? for help
Prelude>~/Repositories/prl_tokyo/bigul/SummerSchool16> ghci test.hs
GHCi, version 7.10.3: http://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main                ( test.hs, interpreted )
Ok, modules loaded: Main.
```

If you see the above message, congratulations for your successful installation.

2 Basics

Intuitively, we can think of a bidirectional BiGUL program:

$$bx :: BiGUL\ s\ v$$

as describing how to manipulate a state consisting of a source component of type s and a view component of type v ; the goal is to embed all information in the view to proper places in the source. For each bx , we can run it forwardly by calling *get* and backwardly by calling *put*.

$$\begin{aligned} \textit{get } bx &:: s \rightarrow v \\ \textit{put } bx &:: s \rightarrow v \rightarrow s \end{aligned}$$

where *get bx* is a function mapping a source to a view, while *put bx* accepts an original source and an update view and returns an updated source.

The core of BiGUL consists of a small number of combinators for constructing bidirectional transformation through development of well-behaved putback transformations.

2.1 Skip

The first one is the simple primitive

$$\textit{Skip} :: \textit{BiGUL } s \ ()$$

Its putback semantics is to skip any change on the source for the unit view. One can test this using `ghci` after loading the `test.hs` by

```
*Main> put Skip 5 ()
Right 5
```

which means that *Skip* putbacks the unit view `()` on the source 5 and gets the same source 5. In fact, each well-behaved putback transformation is equipped with a unique *get* for doing forward transformation.

```
*Main> get Skip 5
Right ()
```

It reads that doing forward transformation for *Skip* on the source 5 gives the unit view `()`.

2.2 Replace

The second one is

$$\textit{Replace} :: \textit{BiGUL } s \ s$$

which is to use the view to replace the source:

```
*Main> put Replace 1 100
Right 100
```

which uses the view 100 to replace the source 1 and gets a new source 100.

2.3 Product: Prod

If we want to putback a view pair (v_1, v_2) to a source pair (s_1, s_2) , we may write $bx1 \text{ Prod } bx2$, a product of two putback transformation, to putback v_1 to s_1 with $bx1$ and v_2 to s_2 with $bx2$.

$$\text{Prod} :: \text{BiGUL } s_1 \ v_1 \rightarrow \text{BiGUL } s_2 \ v_2 \rightarrow \text{BiGUL } (s_1, s_2) \ (v_1, v_2)$$

For instance, we can combine *Skip* and *Replace* to putback a view pair to a source pair.

```
*Main> put (Skip 'Prod' Replace) (5,1) (( ),100)
Right (5,100)
```

We may deal with more complicated structure using nested *Prod*:

```
*Main> put ((Skip 'Prod' Replace) 'Prod' Replace) ((5,1),2) ((( ),100),200)
Right ((5,100),200)
```

2.4 Source/View Rearrangement

So far, the source and the view must be of the same structure. What if we wish to putback a pair view (v_1, v_2) to a complicated source, say $((s0, s_1), s_2)$, where we hope to replace s_1 by v_1 and s_2 by v_2 ? BiGUL provide a way of preprocessing of rearranging either the source or the view through a natural transformation τ .

$$\begin{aligned} \$ (rearrS \ [\ \tau :: s_1 \rightarrow s_2 \]) &:: \text{BiGUL } s_2 \ v \rightarrow \text{BiGUL } s_1 \ v \\ \$ (rearrV \ [\ \tau :: v_1 \rightarrow v_2 \]) &:: \text{BiGUL } s \ v_2 \rightarrow \text{BiGUL } s \ v_1 \end{aligned}$$

Returning to the above problem, we can solve it by defining the following putback transformation

$$\begin{aligned} \text{putPairOverNPair} &:: \text{BiGUL } ((s0, s_1), s_2) \ (s_1, s_2) \\ \text{putPairOverNPair} &= \$ (rearrS \ [\ \lambda((s0, s_1), s_2) \rightarrow (s_1, s_2) \]) \text{ Replace} \end{aligned}$$

and have

```
*PBasic> put putPairOverNPair ((5,1),2) (100,200)
Right ((5,100),200)
```

In fact, it is possible to define *putPairOverNPair* by rearranging the view too.

$$\begin{aligned} \text{putPairOverNPair}' &:: \text{BiGUL } ((s0, s_1), s_2) \ (s_1, s_2) \\ \text{putPairOverNPair}' &= \$ (rearrV \ [\ \lambda(v_1, v_2) \rightarrow (((), v_1), v_2) \]) \$ \\ &\quad (\text{Skip 'Prod' Replace}) \text{ 'Prod' Replace} \end{aligned}$$

The mechanism of source/view rearrangement makes it possible to process algebraic data structures such as lists and trees. The following example describes using view v to replace the first element of a nonempty source.

$$\begin{aligned} pHead &:: \text{BiGUL } [s] \ s \\ pHead &= \$ (rearrS \ [\ \lambda(s: -) \rightarrow s \]) \text{ Replace} \end{aligned}$$

```
*PBasic> put pHead [1,2,3,4] 100
Right [100,2,3,4]
```

We may go further to define a general putback transformation to replace the i th element of a source list with a view v .

```
pNth :: ℤ → BiGUL [s] s
pNth i = if i ≡ 0 then pHead
        else $ (rearrS [| λ(x : xs) → (x, xs) |]) $
              $ (rearrV [| λv → (( ), v) |]) $
                Skip 'Prod' pNth (i - 1)
```

```
*PBasic> put (pNth 3) [1..10] 100
Right [1,2,3,100,5,6,7,8,9,10]
```

Note that any putback function is equipped with a *get* function (which may be partial). So for *pNth*, the corresponding *get* function is actually the familiar *take* function.

```
*PBasic> get (pNth 3) [1..10]
Right 4
```

2.5 Case

The *Case* combinator is for case analysis. The basic structure is as follows.

```
Case [ $ (normal [| cond1 :: s → v → Bool |]) (bx1 :: BiGUL s v),
      $ (normal [| cond2 :: s → v → Bool |]) (bx2 :: BiGUL s v),
      ...,
      $ (normal [| condn :: s → v → Bool |]) (bxn :: BiGUL s v),
      $ (adaptive [| cond1' :: s → v → Bool |]) $ (f1 :: s → v → s),
      $ (adaptive [| cond2' :: s → v → Bool |]) $ (f2 :: s → v → s),
      ...,
      $ (adaptive [| condm' :: s → v → Bool |]) $ (fm :: s → v → s)
    ]
  :: BiGUL s v
```

It contains a sequence of cases. For each case, it is either *normal* or *adaptive*. For the normal case, if the condition is satisfied, a corresponding putback transformation is applied. For the adaptive case, if the condition is satisfied, a function is used to update the source with the view so that for the next step one of the normal cases can be applied. Note that if adaptation does not lead the source and the view to a normal case, an error will be reported at runtime.

As a simple example, consider using the view to update all the elements in the source list. To do so, we can use *Case*.

```
replaceAll :: Eq s ⇒ BiGUL [s] s
replaceAll = Case [
```

```

$ (normal [|  $\lambda s\ v \rightarrow \text{length } s \equiv 1$  |]) $
$ (rearrS [|  $\lambda[x] \rightarrow x$  |]) Replace,
$ (normal [|  $\lambda s\ v \rightarrow \text{length } s > 1$  |]) $
$ (rearrS [|  $\lambda(x : xs) \rightarrow (x, xs)$  |]) $
$ (rearrV [|  $\lambda v \rightarrow (v, v)$  |]) $
  Replace 'Prod' replaceAll,
$ (adaptive [|  $\lambda s\ v \rightarrow \text{length } s \equiv 0$  |]) $
   $\lambda s\ v \rightarrow [\perp]$ 
]

```

```

*PBasic> put replaceAll [] 100
Right [100]
*PBasic> put replaceAll [1..10] 100
Right [100,100,100,100,100,100,100,100,100,100]

```

2.6 Dependency: Dep

Sometimes, a view may contain derived value that is computed from other part of the view and not allowed to be changed. For instance, for the view $(x, x + 1)$, the second component can be computed from the first by increasing it by 1. To capture this, BiGUL provides

$$\text{Dep} :: \text{Eq } v' \Rightarrow \text{BiGUL } a\ v \rightarrow (a \rightarrow v \rightarrow v') \rightarrow \text{BiGUL } a\ (v, v')$$

to describe this intention. We may, for example, define

```

replaceAll2 :: BiGUL [Z] (Z, Z)
replaceAll2 = Dep replaceAll ( $\lambda s\ v \rightarrow v + 1$ )

```

to replace all elements of the source by the first component of the view, while the second component of the view is a derived one that should be one bigger than the first one.

```

*PBasic> put replaceAll2 [1..10] (100,101)
Right [100,100,100,100,100,100,100,100,100,100]
*PBasic> put replaceAll2 [1..10] (100,200)
Left dependency mismatch

```

As seen in the last running of *put*, it reports an error because in the view $(100, 200)$, 200 is not one bigger than 100.

2.7 Composition

Composition of two putback transformations follows that of two bidirectional transformations. This is because each putback transformation is essentially a bidirectional transformation.

$$\text{Compose} :: \text{BiGUL } a \ u \rightarrow \text{BiGUL } u \ b \rightarrow \text{BiGUL } a \ b$$

As an example, consider

$$\begin{aligned} \text{pHead2} &:: \text{BiGUL } [[a]] \ a \\ \text{pHead2} &= \text{pHead} \text{ 'Compose' } \text{pHead} \end{aligned}$$

```
*PBasic> put pHead2 [[1,2],[3,4,5],[]] 100
Right [[100,2],[3,4,5],[]]
```

2.8 Utilities

BiGUL provides some useful functions for building putback transformations. One interesting one is *emb* that can safely embed a pair of well-behaved *get* and *put* a putback transformation into our context. *emb* itself is defined as follows.

$$\begin{aligned} \text{emb} &:: \text{Eq } v \Rightarrow (s \rightarrow v) \rightarrow (s \rightarrow v \rightarrow s) \rightarrow \text{BiGUL } s \ v \\ \text{emb } g \ p &= \text{Case } [\$ (\text{normal } [\lambda x \ y \rightarrow g \ x \equiv y]) \$ \\ &\quad \$ (\text{rearrV } [\lambda x \rightarrow (((), x)]) \$ \\ &\quad \quad \text{Dep Skip } (\lambda x \ () \rightarrow g \ x) \\ &\quad , \$ (\text{adaptive } [\backslash _ \rightarrow \text{True }]) \\ &\quad \quad p \\ &\quad] \end{aligned}$$

As an application of *emb*, we may define a useful putback functions for reflecting the sum to a pair.

$$\begin{aligned} \text{distSum} &:: \text{BiGUL } (\mathbb{Z}, \mathbb{Z}) \ \mathbb{Z} \\ \text{distSum} &= \text{emb } g \ p \\ &\quad \text{where } g \ (x, y) = x + y \\ &\quad \quad p \ (x, y) \ v = (v - y, y) \end{aligned}$$

```
*PBasic> put distSum (1,2) 100
Right (98,2)
*PBasic> get distSum (1,2)
Right 3
```

3 Bidirectional Programming on Lists

In this section, we show many list functions can be bidirectionalized using BiGUL. To show the correspondence with the original we prefix the original forward function names with "lens". Note that the forward function can be automatically derived from the new putback transformation by calling *get*.

3.1 lensFoldr

foldr is a simple but useful higher order function on lists.

$$\begin{aligned} \text{foldr } f \ e \ [] &= e \\ \text{foldr } f \ e \ (x : xs) &= f \ x \ (\text{foldr } f \ e \ xs) \end{aligned}$$

Many interesting functions can be defined in *foldr*

$$\begin{aligned} \text{sum} &= \text{foldr } (+) \ 0 \\ \text{map } f &= \text{foldr } (\lambda a \ r \rightarrow f \ a : r) \ [] \\ \text{filter } p &= \text{foldr } (\lambda a \ r \rightarrow \text{if } p \ a \ \text{then } a : r \ \text{else } r) \ [] \\ \text{reverse } p &= \text{foldr } (\lambda a \ r \rightarrow r \ ++ \ [a]) \ [] \\ \text{sort} &= \text{foldr } \text{insert} \ [] \end{aligned}$$

First, we develop a putback transformation for *foldr*.

$$\begin{aligned} \text{lensFoldr} &:: \text{BiGUL } (a, b) \ b \rightarrow (b \rightarrow \text{Bool}) \rightarrow \text{BiGUL } ([a], b) \ b \\ \text{lensFoldr } bx \ pv &= \\ &\text{Case } [\$ (\text{adaptive } [\lambda(x, y) \ v \rightarrow pv \ v \wedge \text{length } x \neq 0 \] \ \$ \\ &\quad \lambda(x, y) \ v \rightarrow ([], y) \\ &\quad , \$ (\text{normalS } [\lambda(s, e) \rightarrow \text{length } s \equiv 0 \] \ \$ \\ &\quad \quad \$ (\text{rearrV } [\lambda v \rightarrow (((), v)) \] \ \$ \\ &\quad \quad \quad \$ (\text{update } [p \mid (((), v)) \] \ [p \mid ((), v)] \ [d \mid v = \text{Replace} \]]) \\ &\quad , \$ (\text{normalSV } [p \mid - \] \ [p \mid - \] \ \$ \\ &\quad \quad \$ (\text{rearrS } [\lambda((x : xs), e) \rightarrow (x, (xs, e)) \] \ \$ \\ &\quad \quad \quad (\text{Replace 'Prod' lensFoldr } bx \ pv) \ \text{'Compose' } bx \\ &\quad \quad \quad \] \\ &\quad \] \end{aligned}$$

lengFoldr *bx* *pv* is to synchronize source (xs, v) with view v' . If v' satisfies *pv*, it will stop. Otherwise, it will recursively apply *bx* rightwards over the elements of *xs*.

$$\begin{aligned} \text{lensReverse} &:: \text{BiGUL } [a] \ [a] \\ \text{lensReverse} &= \$ (\text{rearrS } [\lambda s \rightarrow (s, []) \] \ \$ \\ &\quad \text{lensFoldr lensSnoc null} \\ \text{lensSnoc} &:: \text{BiGUL } (a, [a]) \ [a] \\ \text{lensSnoc} &= \text{Case } [\$ (\text{normal } [\lambda s \ v \rightarrow \text{length } v \equiv 1 \] \ \$ \\ &\quad \quad \$ (\text{rearrV } [\lambda[v] \rightarrow (v, []) \] \ \text{Replace}, \\ &\quad \$ (\text{normal } [\lambda(-, s) \ v \rightarrow \text{length } s > 0 \] \ \$ \\ &\quad \quad \$ (\text{rearrS } [\lambda(x, y : ys) \rightarrow (y, (x, ys)) \] \ \$ \\ &\quad \quad \quad \$ (\text{rearrV } [\lambda(v : vs) \rightarrow (v, vs) \] \ \$ \\ &\quad \quad \quad \text{Replace 'Prod' lensSnoc}, \\ &\quad \$ (\text{adaptive } [\lambda(-, s) \ v \rightarrow \text{null } s \] \ \$ \\ &\quad \quad \lambda(x, s) \ - \rightarrow (x, [\perp]) \\ &\quad \] \end{aligned}$$

```

*PList> put lensSnoc (1,[2,3,4]) [10,11,12,13]
Right (13,[10,11,12])
*PList> put lensSnoc (1,[2,3,4]) [10,11,12,13,14]
Right (14,[10,11,12,13])
*PList> put lensSnoc (1,[2,3,4]) [10,11]
Right (11,[10])
*PList> get lensSnoc (100,[1..10])
Right [1,2,3,4,5,6,7,8,9,10,100]

*PList> put lensReverse [1..10] [100..105]
Right [105,104,103,102,101,100]
*PList> put lensReverse [1..10] [100..115]
Left either value mismatch
*PList> get lensReverse [1..10]
*PList> get lensReverse [1..10]
Right [10,9,8,7,6,5,4,3,2,1]

```

3.2 Efficiency Issue: *lensFoldr*

A close look at the definition of *lensFoldr* reveals that it contains many redundant computations because of the use of *Compose* that calls *get* as many times as the number of calls of *Compose*. Put it more concretely,

$$\text{put } (\text{lensFoldr } bx \text{ (const True)}) ([x1, x2, \dots, xn], e) v$$

would call

$$\begin{aligned} &\text{get } (\text{lensFoldr } bx \text{ (const True)}) ([x2, \dots, xn], e), \\ &\dots, \\ &\text{get } (\text{lensFoldr } bx \text{ (const True)}) ([], e) \circ \end{aligned}$$

3.3 LensScanr

3.4 LensMap

$$\begin{aligned} \text{lensMap} &:: \text{BiGUL } a \ b \rightarrow \text{BiGUL } ([a], [b]) \ [b] \\ \text{lensMap } bx &= \text{lensFoldr } bx' \ (\lambda v \rightarrow \text{length } v \equiv 0) \\ &\quad \textbf{where } bx' = \$ (\text{rearrV } [\mid \lambda(v : vs) \rightarrow (v, vs) \mid]) \$ \\ &\quad \quad bx \text{ 'Prod' Replace} \end{aligned}$$

For testing, we define

$$\begin{aligned} \text{dec1} &:: \text{BiGUL } \mathbb{Z} \ \mathbb{Z} \\ \text{dec1} &= \text{emb } g \ p \\ &\quad \textbf{where } g \ s = s + 1 \\ &\quad \quad p \ s \ v = v - 1 \end{aligned}$$


```
*PList> put (lensMap dec1) ([0..10],[]) [100..110]
Right ([99,100,101,102,103,104,105,106,107,108,109],[])
*PList> get (lensMap dec1) ([1..10],[])
Right [2,3,4,5,6,7,8,9,10,11]
*PList> put (lensMap dec1) ([0..10],[]) [100]
Right ([99],[])
*PList> put (lensMap dec1) ([0..10],[]) [100..111]
Right ([99,100,101,102,103,104,105,106,107,108,109],[111])
```

4 Bidirectional Programming on Sets

References